Rainbow Run — Analytics Event Playbook

What Dub & PostHog record, scenario by scenario

Engine · TeX backend

June 27, 2026

1 How to read this

Every player interaction leaves a trail across *three* layers. This playbook walks real Rainbow Run journeys and shows exactly what each layer captures at each step — so you know what data you'll have to answer "what's working?".

Layer	Records	Answers
Dub	the **click** on a short's link — device, geo, referrer, `dub_id`	which *short* drives traffic; (phase 2) click→conversion
PostHog	the **in-game funnel** — every event below, with `utm_*`/`dub_id` attached, + session replay	where players drop, who converts, who returns
Hub/`api/track`	a **server-side count** of the same events (ad-blocker-proof)	the true denominator; the game-design eval signal

The thread that ties it together: a short's Dub link lands on `/go/rainbow-run?utm_content=L1-sunbeam-meadow` (Dub appends `dub_id`); the game reads those off the URL and stamps them onto *every* PostHog event and Hub payload. So every chart can be sliced by **which short the player came from**.

2 The event vocabulary (grounded in the real game)

Rainbow Run is 5 weather biomes — *Sunbeam Meadow, Drizzle Dunes, Rainbow Bridge, Frostspark Peaks, Thunder Heights* — played by *Labooboo & Siya*. The trackable signals:

Event	Fires when	Properties (the interesting ones)
`game_open`	first load	utm_source/medium/**content**, dub_id, $device_type
`level_start`	a biome (re)loads	level (1–5), **biome**
`coin_collect`	a coin is grabbed	level, coins_so_far, x, y
`weather_shift`	biome weather cycles (sun→rainbow, →storm …)	level, weather
`player_death`	a death	level, biome, **cause** (fall/lagoon/enemy/crumble/ice), x
`level_complete`	the goal is reached	level, duration_ms, deaths, coins
`game_complete`	Thunder Heights cleared	total_time_ms, total_deaths
`player_idle` / `player_quit`	no input >Ns / tab closed	level, x, ms

3 Scenarios — how it actually plays out

1 · The full conversion (the happy path)

A player on their phone taps the *Thunder Heights* YouTube Short, plays from the start, and finishes all five biomes.

1. taps `dub.sh/PPDuC2r` → **Dub** logs a **click** (device=Mobile, geo, ref=youtube, mints `dub_id`)
2. redirect hits `/go/rainbow-run?utm_content=L5-thunder-heights` → **Hub** logs `click rainbow-run|youtube`
3. game loads → **PostHog** `$pageview`, then `game_open` (carrying utm + dub_id + device)
4. `level_start` L1 Sunbeam Meadow → 7× `coin_collect` → `weather_shift` (sun→rainbow) → `level_complete` (0 deaths)
5. L2→L4 …one `player_death` (cause=`crumble`) on Frostspark Peaks, respawns, `level_complete`
6. clears L5 → `game_complete` (total_deaths=1). **Hub** mirrors every event server-side.

What **PostHog** stores for step 3 (every later event inherits the registered attribution):

```
{ "event": "game_open",
  "properties": { "utm_source":"youtube", "utm_medium":"short",
    "utm_content":"L5-thunder-heights", "dub_id":"pPDuC2r_x9",
    "$device_type":"Mobile", "$current_url":".../?utm_content=L5-thunder-heights" } }
```

and the matching **Hub** call (ad-blocker-proof, same event, server-side):

```
POST /api/track {"game":"rainbow-run","event":"game_open","source":"youtube"}
```

This player counts as a full conversion under `utm_content=L5` — they land in the funnel's bottom step *and* in "completion rate by short" for the Thunder Heights short.

2 · The top-of-funnel bounce (click, no play)

Taps the short, the page opens, but they close the tab before moving.

- **Dub** **click** , **Hub** `click` — but **PostHog** sees only `$pageview` + maybe `game_open`, then `$pageleave`. No `level_start`.

What it tells you: the funnel drops hard at `game_open` → `level_start`. If one short bounces far worse than others (slice by `utm_content`), the *landing* is the problem — slow load, wrong expectation set by the short — not the game.

3 · The struggler (a difficulty signal)

Comes from the *Drizzle Dunes* short, hits the ice + conveyor stretch, dies four times, quits.

```
player_death { level:2, biome:"Drizzle Dunes", cause:"ice", x:520 }
player_death { level:2, biome:"Drizzle Dunes", cause:"fall", x:540 } // x4, clustered at the belt
player_idle { level:2, x:300, ms:9000 }
player_quit { level:2, x:300 } // no level_complete for L2
```

What it tells you: `deaths-by-level` shows a spike on L2 at `x≈520` (the ice belt). Open the **PostHog** **session replay** of that play to watch it; the death cluster is the difficulty heatmap. This is the loop back into game design — widen the belt, slow the conveyor — and the same event feeds the engine's reach/difficulty eval.

4 · The returner (retention by short)

Day 1: arrives from the *Sunbeam Meadow* short, plays L1–L2, leaves. Day 3: opens the game directly (`utm_source=direct`) and plays L3.

- **PostHog** ties both visits to one `distinct_id`; the day-1 `utm_content` stays on the person as a property.
- **PostHog** **Retention** shows a Day-3 return, attributable to the *Sunbeam Meadow* short.

What it tells you: which shorts bring *sticky* players, not just clicks. A short with fewer but higher-retention plays can beat a viral-but-bouncy one.

5 · Mobile vs desktop (a device gap)

A TikTok short sends mostly mobile traffic; those players drop at `level_start` far more than desktop.

device	game_open	level_complete	completion %
Desktop	120	54	45.0
Mobile	300	39	13.0

What it tells you: `convert-by-device` exposes a touch-controls problem. Fixes: tune the on-screen controls, or use **Dub** *device targeting* to route mobile to a touch-optimized entry.

6 · The ad-blocked player (why the Hub backstop exists)

A player on uBlock: the **PostHog** `phc_` ingestion request is blocked, so *zero* client events arrive — but the same-origin **Hub** `/api/track` calls still land.

	PostHog (client)	**Hub** (server)
plays counted	70	100

What it tells you: reconciling the two sizes your blind spot (here ≈30% of short-form, mobile-heavy traffic). **PostHog** is your rich funnel; **Hub** is the honest denominator.

4 What you can now answer (scenario → insight → query)

Question	PostHog surface	HogQL (`scripts/posthog-query.mjs`)
Which short converts best?	Funnel, breakdown `utm_content`	`--canned funnel-by-short`
Where do players die?	Trends/Paths + session replay	`--canned deaths-by-level`
Mobile vs desktop?	Funnel, breakdown $device_type	`--canned convert-by-device`
Which short retains?	Retention, breakdown `utm_content`	persons + first-touch query
Who converts (clusters)?	Funnel **correlation** + behavioral cohorts	`GROUP BY` on segments
Ad-block blind spot?	— (compare to Hub)	hub `/api/funnel` vs PH counts

Status: the read path is live (the `posthog-query.mjs` tool + your personal key already query project 484401). The events above start flowing the moment Rainbow Run is instrumented (see `docs/RAINBOW-RUN-ANALYTICS-BRIEF.md`) — then every scenario here becomes a real chart.